Woxsen University

School of Technology



PROJECT BASED LEARNING REPORT

Design and Implementation of student skill exchange platform

Course Code: 25TU03MJC0
Course Name: Web Technologies
Semester: 3rd
Academic Year: 2026-2027

Submitted by:

1. Zarina Yesmin Barbhuiya – 25WU0104033
2. Masira Khalil - 25WU0104025
3. Diya Bhattacharya - 25WU0103015
4. Baseerah Mansoor - 25WU0103009

Under the guidance of:

Dr. Steven Raj

Assistant Professor, School of Technology

October, 2026

DECLARATION

We hereby declare that the Project Title "***Design and Implementation of student skill exchange platform***" submitted by us is an original work carried out under the guidance of the faculty mentioned above.

We confirm that the system design, implementation logic, and analysis presented in this report are original contributions. This work has not been submitted, either in part or in full, for the award of any other degree, diploma, fellowship, or similar academic qualification at this university or any other educational institution.

Furthermore, I/we attest that all external sources of information utilized during the development of this web application—including academic literature, technical documentation, third-party software libraries, and open-source code—have been fully and properly acknowledged. All references have been strictly cited in accordance with IEEE formatting and Woxsen University's referencing standards.

Finally, we declare that this manuscript has been subjected to rigorous plagiarism checks, and the similarity index falls entirely within the permissible limits prescribed by the academic integrity guidelines of the institution.

Place: Hyderabad
Date: 08/10/2026

1. Zarina Yesmin Barbhuiya – 25WU0104033

Signature:

2. Masira Khalil - 25WU0104025

Signature:

3. Diya Bhattacharya - 25WU0103015

Signature:

4. Baseerah Mansoor - 25WU0103009

Signature:

CERTIFICATE

This is to formally certify that the Project-Based Learning (PBL) report titled **"*Design, Implementation and Performance Analysis of a Student's Skill Exchange Platform*"** is a bonafide and original record of work collaboratively carried out by:

- **Zarina Yesmin** (Register No: 4033)
- **Masira Khalil** (Register No: 4025)
- **Diya Bhattacharya** (Register No: 3015)
- **Baseerah Mansoor** (Register No: 3009)

This project was successfully completed as part of the academic curriculum for the **B.Tech – Computer Science and Engineering** program during the Academic Year 2026-27.

I confirm that the students have developed this web application and compiled this technical report under my direct supervision and guidance. To the best of my knowledge, the project satisfies the practical requirements of the Web Technologies course and meets the prescribed academic and technical standards of the university.

Guide Signature

Dr. Steven Raj
Assistant Professor, School of Technology

ACKNOWLEDGEMENT

The successful completion of this Project-Based Learning (PBL) report on the "Student's Skill Exchange Platform" would not have been possible without the support, guidance, and encouragement of several individuals and institutional bodies. We would like to take this opportunity to express our profound gratitude to all who contributed to this endeavor.

First and foremost, we extend our sincere appreciation to Woxsen University and the School of Technology for providing a dynamic academic environment and state-of-the-art infrastructure that fostered our learning and project development. We are particularly grateful to the Head of the Department of Computer Science and Engineering for integrating such practical, problem-based learning elements into the Web Technologies curriculum, which allowed us to explore real-world full-stack development.

Our deepest gratitude goes to our faculty guide, [Insert Guide's Name Here], for their unwavering support, technical mentorship, and constructive feedback throughout the project lifecycle. Their expertise in web architecture and database management was instrumental in helping us navigate complex coding challenges and shape the final platform.

We would also like to thank our fellow classmates and peers whose insightful discussions and willingness to test our prototype significantly improved the user experience and functionality of our application.

Finally, we acknowledge the collective effort, dedication, and synergy of our project team. The successful integration of HTML, CSS, JavaScript, PHP, Servlets, JSP, and MySQL into a unified WebRTC-enabled platform is a testament to our shared commitment to collaborative learning.

1. ABSTRACT

University ecosystems are incredibly rich in intellectual capital, yet the distribution of this knowledge often remains heavily localized and siloed. Students frequently encounter significant academic hurdles when attempting to master complex, multidisciplinary subjects. While traditional professional tutoring provides a reliable solution, it is often financially prohibitive for the average learner. Simultaneously, static digital learning platforms and isolated online tutorials lack the dynamic, interactive feedback loop essential for deep technical comprehension. Consequently, a vast reservoir of peer talent goes untapped, leaving students without a structured, trusted environment to organically collaborate and solve problems together.

To dismantle these educational barriers, this project introduces the Student's Skill Exchange Platform, a zero-cost, centralized web application explicitly designed to democratize peer-to-peer learning. Operating entirely on a mutually beneficial barter ecosystem, the platform empowers students to leverage their distinct academic strengths without financial transaction. Users can catalog their specific proficiencies—ranging from frontend user interface design and Java Object-Oriented Programming to relational database architecture—and seamlessly trade these skills for guidance in disciplines where they require support. A defining feature of this ecosystem is the integrated, real-time WebRTC video exchange dashboard. This advanced functionality allows users to transition instantly from navigating a unified skill marketplace to engaging in live, one-on-one collaborative tutoring sessions, effectively bridging the gap between asynchronous requests and synchronous learning.

Engineered to comprehensively fulfill the rigorous requirements of the Web Technologies curriculum, the system is built upon a highly responsive, multi-tier architecture. The client-side presentation layer utilizes modern HTML5, CSS3, and JavaScript, delivering an intuitive interface fortified by strict client-side form validation and interactive DOM manipulation. The robust application tier manages core business logic, secure user authentication, and dynamic content generation through a synergistic integration of PHP, Java Servlets, and JSP. Furthermore, the platform employs XML for standardized data exchange and categorization, while relying on secure JDBC connectivity to a persistent MySQL database. This ensures efficient CRUD (Create, Read, Update, Delete) operations, reliable data storage, and robust session tracking across the entire application lifecycle.

Ultimately, this project transcends the boundaries of a standard academic exercise. By synthesizing full-stack web development frameworks into a practical, scalable application, the Student's Skill Exchange Platform addresses a tangible campus challenge. It redefines the collaborative learning experience, transforming fragmented student expertise into a cohesive, accessible marketplace that fosters academic excellence, technical proficiency, and community-driven growth.

TABLE OF CONTENTS

DECLARATION ii

CERTIFICATE iii

ACKNOWLEDGEMENT iv

1. ABSTRACT .....v-vi

LIST OF FIGURES.....ix

FIST OF TABLES.....x

LIST OF LISTINGS.....x

2. INTRODUCTION .....[Page 1-3]

2.1 Background

2.2 Motivation & Relevance to Real-World Problem

2.3 Scope & Organization of the Report

3. PROBLEM STATEMENT & OBJECTIVES .....[Page 4-6]

3.1 Problem Definition

3.2 Core Objectives

3.3 Constraints & Assumptions

4. LITERATURE REVIEW .....[Page 7-10]

4.1 Massive Open Online Courses (MOOCs)

4.2 Freelance and Professional Micro-Task Marketplaces

4.3 University-Specific Learning Management Systems (LMS) Forums

4.4 Global Q&A Communities and Knowledge Repositories

4.5 Digital Timebanking and Reciprocal Service Models

4.6 Synchronous Collaborative Tooling

4.7 Comparison of Existing Platforms vs. Proposed System

5. METHODOLOGY & SYSTEM DESIGN .....[Page 11-17]

5.1 Overall System Architecture

5.2 Process Flowchart and Development Lifecycle

5.3 Algorithm Explanation (Matchmaking & WebRTC)

5.3.1 Data-Driven Matchmaking Algorithm

5.3.2 WebRTC Peer-to-Peer Connection Logic

5.4 Tools & Technologies Stack

5.5 Use-Case & Data Flow Diagram

5.6 Database Schema and Entity Relationships

6. IMPLEMENTATION .....[Page 18-26]

6.1 Step-by-Step Module Implementation

6.2 Dataset Description and Relational Architecture

6.3 Hardware and Software Environment

6.4 Algorithmic Execution and Logic Flow

6.5 Local Deployment and Execution Flow

7. RESULTS & DISCUSSION .....[Page 27-29]

7.1 Functional Testing and Output Validation

7.2 Performance Evaluation

7.3 Discussion of Findings

7.4 Limitations and Future Scope

8. CONCLUSION & FUTURE SCOPE .....[Page 30-31]

8.1 Conclusion

8.2 Future Scope

REFERENCES .....[Page 32-33]

LIST OF TABLES

- Figure 5.1: Multi-tier system architecture flow
- Figure 5.2: Project Development Lifecycle Flowchart
- Figure 5.3: System Use-Case Diagram
- Figure 6.1: Student Registration and Institutional Domain Validation Interface
- Figure 6.2: Dynamic JSP Marketplace Dashboard
- Figure 6.3: Active WebRTC Video Platform
- Figure 6.4: User Logout and Session State Termination
- Figure 6.5: Normalized Relational Database Schema and Entity Relationships
- Figure 6.6: Database Schema Implementation in phpMyAdmin

LIST OF TABLES

- Table 4.1: Comparison of Existing Digital Learning Platforms vs. Proposed System
- Table 7.1: Functional Test Cases and System Validation Outcomes

LIST OF LISTINGS (ALGORITHMS & CODE)

- Listing 5.1: Java Servlet and JDBC Data Retrieval Logic
- Listing 5.2: JavaScript WebRTC and Peer.JS Signaling Logic
- Listing 6.1: Complete Authentication, Registration, and Session Ingestion Sequence
- Listing 6.2: Dynamic Marketplace Population via Java Servlet & JDBC
- Listing 6.3: WebRTC Peer-to-Peer Media Session Handshake

2. INTRODUCTION

2.1 Background

The landscape of modern higher education is increasingly defined by the integration of collaborative technologies and digital ecosystems. As universities transition from traditional, lecture-heavy models toward interactive and practical paradigms, peer-to-peer (P2P) learning has emerged as a cornerstone of academic success. Within any rigorous technical institution, the student body represents a massive, untapped reservoir of highly specialized knowledge. Students across various disciplines—such as Computer Science, Data Science, and Information Technology—develop niche proficiencies in programming languages, algorithmic design, and software architecture. However, these academic ecosystems frequently suffer from structural silos. A student excelling in Java Object-Oriented Programming might struggle significantly with frontend interface design, while a peer in the same cohort possesses the exact opposite skill set.

Historically, bridging this gap relied on informal study groups or university-sponsored tutoring centers, which are often limited by scheduling constraints and resource availability. The advent of dynamic web technologies has provided the foundational infrastructure to digitize and scale this collaborative process. Modern web development frameworks allow for the creation of centralized, responsive platforms where data can be shared asynchronously and connections can be forged in real-time.

The "Student's Skill Exchange Platform" is conceptualized against this backdrop. It leverages the comprehensive spectrum of the Web Technologies curriculum—spanning from client-side markup and scripting to robust server-side processing and database management—to architect a digital marketplace. By utilizing HTML, CSS, and JavaScript for the frontend, coupled with PHP, Java Servlets, and JSP for backend logic, the project translates theoretical web architecture into a tangible, high-impact community tool. Furthermore, the integration of structured data formats like XML and relational databases via MySQL ensures that the system is scalable, persistent, and capable of managing complex user interactions.

2.2 Motivation & Relevance to Real-World Problem

The primary motivation for this project stems from a deeply entrenched real-world problem observed within university campuses: the financial and logistical barriers to personalized academic assistance. When students encounter technical roadblocks—such as debugging complex code or understanding database normalizations—they typically turn to premium online courses (like Udemy or Coursera) or professional freelance tutors. These options are often financially prohibitive for the average university student. Conversely, free digital resources and static tutorial videos lack the interactive, iterative feedback loop required for deep technical comprehension.

This project introduces a zero-cost, barter-based economy specifically tailored for the academic environment. The platform operates on a simple but powerful premise: every student has something valuable to teach and something critical to learn. By enforcing a strict skill-for-skill exchange mechanism, the platform removes financial transactions entirely, fostering an inclusive community where knowledge is the only currency.

The relevance of this project is further elevated by its prototype implementation of a real-time WebRTC video exchange dashboard. Modern remote learning has highlighted the necessity of synchronous communication. A static request board is insufficient for teaching complex technical concepts; students need to screen-share, converse, and collaboratively write code. By integrating a live 1-on-1 video room directly into the web interface using JavaScript and PeerJS, the platform bridges the gap between finding a peer and actively learning from them.

From an academic perspective, this project is highly relevant to the Project-Based Learning (PBL) mandate. It forces the transition from isolated coding exercises to full-stack system integration. Developing this platform requires solving real-world software engineering challenges, including secure session management, state preservation across dynamic JSP pages, asynchronous data handling, and cross-browser UI compatibility, thereby directly satisfying the overarching objectives of the Web Technologies syllabus.

2.3 Scope & Organization of the Report

Scope of the Project: The scope of the Student's Skill Exchange Platform encompasses the end-to-end design, development, and deployment of a multi-tier web application. The system is scoped to serve authenticated university students, providing them with a secure environment to manage personalized profiles, catalog their offered and requested skills, and initiate peer connections. Functionally, the prototype includes client-side form validation, secure login mechanisms utilizing PHP sessions and cookies, a dynamic skill marketplace rendered via Servlets and JSP, and a live peer-to-peer video communication interface. The database scope includes a fully normalized MySQL schema connected via JDBC to handle persistent CRUD (Create, Read, Update, Delete) operations for users and exchange requests.

Organization of the Report:

To provide a comprehensive overview of the system's lifecycle, this report is systematically divided into the following chapters:

- **Chapter 1: Abstract** provides a high-level executive summary of the project's purpose, architecture, and outcomes.
- **Chapter 2: Introduction** outlines the foundational background, primary motivations, and the boundaries of the proposed system.
- **Chapter 3: Problem Statement & Objectives** explicitly defines the campus knowledge gap and sets measurable technical goals aligned with the PBL criteria.

- **Chapter 4: Literature Review** analyzes existing digital learning environments, MOOCs, and freelance platforms, highlighting their limitations for student bartering through comparative analysis.
- **Chapter 5: Methodology & System Design** details the architectural blueprint of the platform, featuring multi-tier diagrams, database schemas, and process flowcharts.
- **Chapter 6: Implementation** documents the active development phase, showcasing the integration of HTML, CSS, JavaScript, PHP, JSP, and WebRTC logic, supported by interface screenshots.
- **Chapter 7: Results & Discussion** evaluates the platform's performance, testing outcomes, and the success of the integrated video exchange feature.
- **Chapter 8: Conclusion & Future Scope** summarizes the learning outcomes achieved and proposes future enhancements for platform scalability.

3. PROBLEM STATEMENT & OBJECTIVES

3.1 Problem Definition

The modern university ecosystem is characterized by a highly diverse student body, possessing a wide array of specialized technical and academic skills. However, the distribution of this knowledge remains heavily fragmented and localized. When a student faces an academic hurdle—such as grasping complex Object-Oriented Programming principles in Java or designing a normalized relational database—they are typically forced to choose between two suboptimal paths: seeking professional tutoring, which is often financially inaccessible, or relying on static online tutorials, which lack personalized, real-time feedback.

Currently, there is no centralized, university-specific digital infrastructure that facilitates the organic, cashless exchange of academic skills between peers. Informal networks, such as campus bulletin boards or decentralized messaging groups (e.g., WhatsApp or Discord cohorts), are highly inefficient for matching specific academic needs. These ad-hoc methods lack searchability, profile verification, and, most importantly, an integrated environment for the actual learning process to take place.

Furthermore, the existing digital platforms designed for freelancing or online learning (such as Fiverr or Coursera) operate on a rigid monetary transaction model, completely ignoring the potential of a skill-barter economy. This creates a significant "knowledge silo" effect on campus, where immense peer talent remains underutilized simply because students lack a structured medium to connect and collaborate.

From a technical standpoint, addressing this gap requires engineering a sophisticated, multi-tier web application capable of handling user authentication, dynamic data matching, and live communication. The core technical problem lies in designing a system that seamlessly integrates a persistent relational database (MySQL) with a dynamic frontend, while also establishing a synchronous, low-latency communication channel (WebRTC) to allow students to teach and learn directly within the browser, without requiring third-party video conferencing software.

3.2 Core Objectives

To address the aforementioned challenges, this Project-Based Learning (PBL) initiative focuses on designing and implementing a robust Student's Skill Exchange Platform. The primary objectives are formulated to directly map the overarching goals of the system to the technical requirements of the Web Technologies curriculum.

- To Develop a Multi-Tier Full-Stack Web Architecture

The fundamental objective is to build a responsive, intuitive web interface using HTML, CSS, and JavaScript, while managing server-side operations through PHP, Java Servlets, and

dynamic JSP pages. This ensures a seamless user experience from registration to the active dashboard interface.

- To Implement a Zero-Cost Barter Matchmaking Mechanism

The system aims to categorize and match users based strictly on their offered and requested competencies (e.g., offering Data Science tutoring in exchange for Advanced CSS guidance). This objective includes utilizing XML for structured skill categorization and implementing search logic to match compatible peers dynamically.

- To Integrate Real-Time WebRTC Video Communication

A critical objective of the platform is to transcend asynchronous messaging. By integrating the PeerJS library and WebRTC protocols via JavaScript, the platform will feature an embedded 1-on-1 video and audio exchange room. This allows peers to seamlessly transition from finding a match to conducting a live, interactive learning session within the same browser window.

- To Ensure Secure Data Persistence and Session Management

The project aims to construct a highly secure backend infrastructure. This involves designing a normalized MySQL database schema to store user credentials and skill requests securely. Furthermore, the objective includes utilizing JDBC (Java Database Connectivity) for reliable data retrieval and employing PHP and Servlets for secure user authentication, cookie management, and session tracking.

3.3 Constraints & Assumptions

Assumptions:

- Hardware Availability: It is assumed that all participating users possess a standard computing device equipped with a functional webcam and microphone to utilize the WebRTC video exchange feature.
- Browser Compatibility: It is assumed that users will access the platform using modern, WebRTC-compliant web browsers (such as Google Chrome, Mozilla Firefox, or Microsoft Edge) that support the latest JavaScript ES6 standards and HTML5 multimedia elements.
- Institutional Identity: It is assumed that all registered users are active students, validated through an institutional email domain (e.g., @woxsen.edu.in), ensuring a trusted and secure campus community environment.

Constraints:

- Financial Gateway Absence: As a strict constraint, the platform is restricted entirely to a barter economy. No payment gateways or monetary transaction modules will be implemented, limiting interactions strictly to skill-for-skill trades.

- Signaling Server Reliance: For the prototype's WebRTC integration, the system relies on the free, public PeerJS signaling server to establish initial peer connections. Under high concurrent user loads, this third-party reliance may introduce minor latency in call initiation compared to a dedicated, self-hosted TURN/STUN server.
- Scope of Deployment: The current iteration of the project is constrained to a web-based application utilizing XAMPP/Tomcat localhost environments for testing. It is not currently optimized for compilation into a native mobile application, though the frontend is designed to be mobile-responsive.

4. LITERATURE REVIEW

The evolution of digital learning and peer-to-peer collaboration has been extensively documented in recent academic literature and technological implementations. To design an effective Student's Skill Exchange Platform, it is crucial to analyze existing educational frameworks, professional marketplaces, and communication platforms. This review examines six prominent categories of existing related works, evaluating their architectures, pedagogical benefits, and inherent limitations regarding zero-cost academic skill bartering.

4.1 Massive Open Online Courses (MOOCs)

Massive Open Online Courses, such as Coursera, edX, and Udacity, have fundamentally transformed digital education by providing access to high-quality, structured curriculum from top-tier universities. From a technical architecture standpoint, these platforms utilize complex content delivery networks (CDNs) and scalable databases to serve asynchronous video content to millions of concurrent users. However, literature highlights a significant pedagogical limitation: the lack of personalized, real-time peer interaction. While MOOCs excel at one-to-many knowledge broadcasting, they operate on a highly asynchronous communication paradigm. A student struggling with a specific dataset or a niche Java configuration error cannot easily request immediate, 1-on-1 assistance. Furthermore, while the content is often free to audit, verified assessments and interactive grading require monetary payment, placing a financial barrier on complete academic engagement. The proposed Skill Exchange Platform addresses this gap by shifting the paradigm from one-to-many broadcasting to one-to-one, synchronous peer bartering without financial constraints.

4.2 Freelance and Professional Micro-Task Marketplaces

Platforms such as Fiverr, Upwork, and Freelancer represent the pinnacle of digital skill marketplaces. These systems utilize sophisticated algorithmic matchmaking, relational database structures for portfolio management, and secure third-party payment gateways to facilitate professional exchanges. While they are highly effective for commercial transactions, their fundamental economic model makes them unsuitable for everyday university peer learning. The core barrier is monetization; these platforms operate strictly on a fiat currency model, which is often inaccessible to students seeking help with routine academic assignments. Additionally, these platforms are optimized for project outsourcing rather than collaborative learning—the goal is product delivery, not pedagogical comprehension. By stripping away the payment gateway and substituting it with a strict skill-for-skill database validation (matching a "requested" skill with an "offered" skill), our system repurposes the marketplace architecture specifically for academic collaboration.

4.3 University-Specific Learning Management Systems (LMS) Forums

Academic institutions universally employ Learning Management Systems like Canvas, Blackboard, or Moodle, which include integrated discussion forums (e.g., Piazza). These tools

are highly secure, institutionally verified, and effectively organize communication by course module. Despite these advantages, literature points out that LMS forums are inherently text-heavy and asynchronous. Students post questions and wait hours or days for a response from a peer or Teaching Assistant. The communication loop is disconnected, making it incredibly difficult to troubleshoot dynamic problems like runtime errors or complex mathematical derivations. Moreover, these systems lack cross-disciplinary integration; a Computer Science student cannot easily post a request in the Business School's forum. The proposed system solves this by acting as a centralized, cross-departmental hub, while upgrading the communication layer from asynchronous text to live WebRTC video integration, enabling immediate, visual problem-solving.

4.4 Global Q&A Communities and Knowledge Repositories

Platforms like Stack Overflow, Quora, and GitHub Discussions serve as massive repositories of crowdsourced technical knowledge. They utilize advanced indexing, tag-based categorization (similar to the XML schema proposed in this project), and community-driven reputation systems (upvoting/downvoting) to surface accurate answers. While Stack Overflow is indispensable for modern programming, it suffers from a high barrier to entry for beginners and a notoriously strict moderation culture that can intimidate novice students. Furthermore, it is a transactional information retrieval system, not a mentorship platform. A student receives an answer to a highly specific code snippet but does not receive the foundational tutoring necessary to understand the underlying architecture. Our Skill Exchange Platform shifts the focus from isolated Q&A to sustained, interactive peer mentorship, fostering deeper comprehension through live conversation.

4.5 Digital Timebanking and Reciprocal Service Models

The concept of "timebanking"—where hours of service are recorded as a currency—has been explored in various community-building applications. In these systems, a user earns a "time credit" by providing an hour of service, which they can later spend to receive an hour of service from someone else. While conceptually aligned with our goals, most existing timebanking platforms are generalized community boards (e.g., trading plumbing services for gardening) and lack the specialized technical infrastructure required for academic tutoring. They do not feature integrated coding environments, live video rooms, or specific academic categorizations. By merging the reciprocal philosophy of timebanking with a modern web technology stack (PHP/Servlets for session management and WebRTC for communication), our platform modernizes the barter system specifically for the technical demands of a university cohort.

4.6 Synchronous Collaborative Tooling

The rise of remote work has led to the development of powerful synchronous collaboration tools, ranging from video conferencing software (Zoom, Microsoft Teams) to live collaborative coding environments (Replit, CodePen). These tools perfectly execute real-time, low-latency interaction. However, they are strictly communication or execution mediums; they lack built-

in marketplace or matchmaking capabilities. A student cannot use Zoom to *find* a Java tutor; they can only use it to communicate *after* they have independently found one. Our proposed system bridges this technological divide by integrating the discovery phase (the JSP/MySQL-driven marketplace dashboard) with the execution phase (the embedded PeerJS/WebRTC video room) into a single, seamless Single-Page Application (SPA) architecture.

4.7 Comparison of Existing Platforms vs. Proposed System

The following table synthesizes the literature review, highlighting the critical feature gaps in existing platforms that the Student's Skill Exchange Platform is engineered to resolve.

Platform / Category	***Primary Economic Model***	***Matchmaking Mechanism***	***Synchronous Interaction (Live Video)***	***Cross-Disciplinary Academic Focus***	***Major Limitation for Student Use***
MOOCs (Coursera, edX)	Freemium / Paid Certification	Algorithm-driven Course Recommendations	No (Strictly Asynchronous)	Yes	Lacks 1-on-1 peer interaction; financial barriers for assessment.
Freelance (Upwork, Fiverr)	Fiat Currency / Monetary	Search & Bid System	Rare (Usually text/email based)	No (Commercially focused)	Cost-prohibitive for students; focuses on outsourcing, not learning.
LMS Forums (Piazza, Canvas)	Free (Institutionally Funded)	Course-Specific Silos	No (Text-based threads)	No (Restricted by specific class)	High latency in communication; siloed within individual courses.
Q&A Sites (Stack Overflow)	Free (Ad-Supported)	Search & Tag Indexing	No (Asynchronous text)	Mixed (Heavily skewed to tech)	Highly impersonal; no continuous mentorship or live problem-solving.
Collab Tools (Zoom, Replit)	Freemium Subscription	None (Requires external scheduling)	Yes (High-quality Video/Audio)	Yes	Lacks a marketplace to discover peers; strictly a communication tool.

Platform / Category	***Primary Economic Model***	***Matchmaking Mechanism***	***Synchronous Interaction (Live Video)***	***Cross-Disciplinary Academic Focus***	***Major Limitation for Student Use***
Proposed System (Skill Exchange)	Zero-Cost Skill Barter	Dynamic Tagging (Requested vs. Offered)	Yes (Embedded WebRTC)	Yes (Campus-Wide Integration)	Restricted to university network constraints; relies on peer availability.

Table 4.1: Comparison of Existing Digital Learning Platforms vs. Proposed System

The comparison table shows that current platforms do not fully meet the everyday needs of students looking for quick, peer-to-peer help. Commercial websites usually require payment, while typical university forums rely on slow, text-based replies.

Our proposed Student's Skill Exchange Platform solves these issues. By creating a free, barter-based system and adding live WebRTC video calls, this project provides a practical, interactive, and cost-free way for students to connect, share their skills, and learn together on campus.

5. METHODOLOGY & SYSTEM DESIGN

5.1 Overall System Architecture

The Student's Skill Exchange Platform is engineered utilizing a robust, multi-tier web architecture that effectively separates the presentation, application, and data management layers. This modular approach ensures high scalability, maintainability, and seamless integration of the disparate technologies required by the course syllabus, including PHP, Java Servlets, JSP, MySQL, and WebRTC.

- Presentation Layer (Client Tier): The frontend is developed using HTML5, CSS3, and JavaScript. This layer is responsible for rendering the Single-Page Application (SPA) interface, which includes the skill marketplace dashboard and the video exchange room. CSS Grid and Flexbox are employed to create a responsive, modern UI. JavaScript handles dynamic DOM manipulation, client-side form validation, and the initialization of the PeerJS WebRTC library for direct browser-to-browser multimedia streaming.

- Business Logic Layer (Application Tier): This intermediate layer processes user requests and orchestrates data flow. PHP is utilized specifically for secure user authentication, managing login states, and generating session cookies. For complex data retrieval and dynamic dashboard rendering, Java Servlets act as controllers. These Servlets intercept HTTP requests, execute business logic, and forward the processed data to JavaServer Pages (JSP), which dynamically compile the HTML populated with the latest skill requests from the database.

- Data Persistence Layer (Database Tier): The foundational layer consists of a normalized MySQL relational database. It securely stores user credentials, profile information, and the active exchange requests. Java Database Connectivity (JDBC) drivers are employed within the Servlets to execute SQL queries. Additionally, Extensible Markup Language (XML) is utilized to maintain a static, structured catalog of broad academic categories (e.g., Programming, UI/UX Design), providing a standardized taxonomy for the skills database.

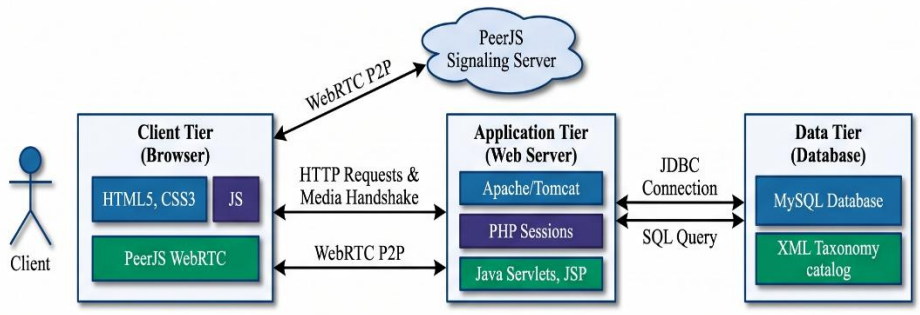


Figure 5.1: Multi-tier system architecture flow

5.2 Process Flowchart and Development Lifecycle

The development methodology followed an iterative System Development Life Cycle (SDLC) tailored for Project-Based Learning (PBL). The project was divided into sequential phases to ensure all syllabus constraints were systematically addressed.

1. Requirement Analysis & Specification: The initial phase involved defining the core constraints: a zero-cost academic barter system requiring secure authentication, dynamic data presentation, and live video interaction. The database schema and XML structure were theoretically drafted during this phase.

2. UI/UX Prototyping: A high-fidelity prototype of the dashboard was constructed using HTML and CSS. The design prioritized a frictionless user experience, ensuring that students could easily navigate from the marketplace grid to the video interface without encountering page reloads.

3. Backend Integration (Database & Servlets): The MySQL database was instantiated locally via XAMPP. JDBC connections were established within Java Servlets to handle CRUD operations. Concurrently, PHP scripts were developed to manage the registration and login pipelines securely.

4. WebRTC Signaling & Video Integration: The most complex phase involved integrating the PeerJS library. This required establishing event listeners in JavaScript to capture the local media stream (camera and microphone) and handle the asynchronous signaling required to connect two distinct browsers via unique Peer IDs.

5. System Testing & Refinement: The integrated system underwent rigorous testing in a localhost environment. Test cases included verifying session persistence, ensuring SQL queries returned accurate matches, and testing the WebRTC connection under simulated multi-user scenarios using browser incognito instances.

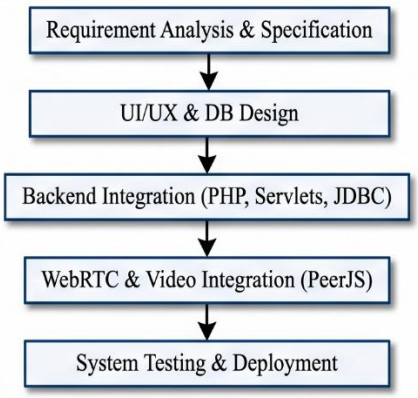


Figure 5.2: Project Development Lifecycle Flowchart

5.3 Algorithm Explanation (Matchmaking & WebRTC)

The core functionality of the platform relies on two distinct algorithmic processes: the data-driven matchmaking logic and the peer-to-peer multimedia routing.

5.3.1 Data-Driven Matchmaking Algorithm:

Unlike commercial platforms that utilize complex machine learning for recommendations, this academic prototype relies on strict Boolean matching facilitated by relational database queries.

1. Data Ingestion: When a student posts a request, the system captures their StudentID, OfferedSkillID, and RequestedSkillID.
2. Query Execution: The Java Servlet triggers a SQL SELECT statement equipped with JOIN operations. The algorithm scans the Exchange_Request table to cross-reference the active user's required skill against the offered skills of all other active users in the database.
3. Result Set Processing: The JDBC ResultSet translates the relational data into Java objects.
4. Dynamic Dispatch: The Servlet packages this list of matched peers as request attributes and uses RequestDispatcher to forward the data to the JSP. The JSP iterates through this list utilizing a for loop, dynamically generating the HTML structure for each peer's user card on the marketplace dashboard.

```
// 1. Establish JDBC Connection
Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);

// 2. Define the SQL Query to fetch active user requests and their skills
String sqlQuery = "SELECT s.Name, s.Department, e.OfferedSkill, e.RequestedSkill,
        "FROM Student s JOIN Exchange_Request e ON s.StudentID = e.StudentID " +
        "WHERE s.StudentID != ?"; // Exclude current user

PreparedStatement pstmt = conn.preparStatement(sqlQuery);
pstmt.setString(1, currentSessionUserId);
ResultSet rs = pstmt.executeQuery();

// 3. Process the ResultSet into a Java List
List<UserRequest> marketplaceData = new ArrayList<>();
while (rs.next()) {
    UserRequest req = new UserRequest();
    req.setName(rs.getString("Name"));
    req.setOffering(rs.getString("OfferedSkill"));
    req.setSeeking(rs.getString("RequestedSkill"));
    req.setPeerId(rs.getString("PeerID"));

    marketplaceData.add(req);
}

// 4. Forward the data to the JSP for dynamic HTML rendering
request.setAttribute("activeRequests", marketplaceData);
RequestDispatcher dispatcher = request.getRequestDispatcher("dashboard.jsp");
dispatcher.forward(request, response);
```

Listing 5.1: Java Servlet and JDBC Data Retrieval Logic

5.3.2 WebRTC Peer-to-Peer Connection Logic:

To facilitate live video exchanges without the latency and server costs associated with traditional video hosting, the system utilizes WebRTC (Web Real-Time Communication) via the PeerJS wrapper.

1. Peer Initialization: Upon loading the dashboard, the local client connects to a public PeerJS signaling server. The server assigns the client a universally unique identifier (UUID).

2. Media Acquisition: The navigator.mediaDevices.getUserMedia API is invoked to request access to the user's local webcam and microphone, binding the resulting media stream to the local HTML <video> element.

3. Call Initiation (The Offer): When the user inputs a target Peer ID and clicks connect, the local client generates a Session Description Protocol (SDP) "Offer" and sends it through the signaling server to the target peer.

4. ICE Candidate Exchange: Simultaneously, both clients begin gathering Interactive Connectivity Establishment (ICE) candidates, determining the optimal network path (via STUN/TURN servers) to bypass NAT/Firewalls and establish a direct connection.

5. Call Acceptance (The Answer): The remote peer receives the offer and, upon user confirmation, generates an SDP "Answer". Once the handshake is complete, the signaling server steps away, and the video/audio streams are transmitted directly between the two browsers.

```
// 1. Initialize connection to the signaling server
const peer = new Peer();
let localStream;

// 2. Request access to the user's camera and microphone
navigator.mediaDevices.getUserMedia({ video: true, audio: true })
    .then(stream => {
        localStream = stream;
        document.getElementById('local-video').srcObject = stream; // Display local video

        // 3. Listen for INCOMING call requests
        peer.on('call', call => {
            // Automatically answer the call and send our local media stream
            call.answer(localStream);

            // Receive and display the remote peer's video stream
            call.on('stream', remoteStream => {
                document.getElementById('remote-video').srcObject = remoteStream;
            });
        });
    })
    .catch(error => console.error("Media access denied:", error));

// 4. Function to initiate an OUTGOING call
function initiateCall(targetPeerId) {
    // Send call request to the target peer's ID along with our local stream
    const call = peer.call(targetPeerId, localStream);
    // Once the target answers, display their video stream
    call.on('stream', remoteStream => {
        document.getElementById('remote-video').srcObject = remoteStream;
    });
}
```

Listing 5.2: JavaScript WebRTC and PeerJS Signaling Logic

5.4 Tools & Technologies Stack

The technology stack was carefully selected to fulfill the comprehensive requirements of the B.Tech Web Technologies syllabus while providing a modern, scalable application.

- HTML5 & CSS3: Provide the semantic structure and visual presentation. Advanced CSS features like variables, flexbox, and grid layouts are used to create the glassmorphism effects and responsive video wrappers.
- JavaScript (ES6): Serves as the backbone of the client-side interactivity. It is exclusively responsible for handling WebRTC events, asynchronous UI updates, and strict form validation before data is submitted to the backend.
- PHP: Deployed specifically for its robust, built-in session management capabilities. It authenticates users against the MySQL database and sets secure cookies to maintain the user's identity across the platform.
- Java Servlets & JSP: Form the core application engine. Servlets provide a secure, multi-threaded environment for processing business logic, while JSP allows for the seamless embedding of Java code within HTML to generate dynamic marketplace content based on database states.
- MySQL: Acts as the primary relational database management system (RDBMS). It is structured using the Third Normal Form (3NF) to ensure data integrity and eliminate redundancy across the Student, Skills, and Requests tables.
- JDBC: The Java Database Connectivity API acts as the bridge between the Servlets and the MySQL database, executing prepared statements to prevent SQL injection vulnerabilities.
- XML & DTD: Extensible Markup Language is utilized alongside a Document Type Definition to explicitly define and validate the hierarchical taxonomy of academic skill categories available on the platform, separating static configuration data from dynamic relational data.

5.5 Use-Case & Data Flow Diagram

The system's usability is designed around a continuous, intuitive loop that takes a student from registration to an active learning session.

System Actors:

- Primary Actor: The Authenticated Student.
- System Actors: The Web Server (Tomcat/Apache) and the Signaling Server (PeerJS).

Primary Use Cases:

1. Authentication: The student inputs credentials. The PHP server validates this against the database and grants access via session tokens.
2. Skill Cataloging: The student navigates the XML-driven skill catalog to select their competencies and requirements.
3. Marketplace Browsing: The student views a dynamically populated JSP dashboard displaying peer requests.
4. Synchronous Connection: The student initiates a WebRTC connection. The system negotiates the SDP/ICE handshake.
5. Collaborative Session: The students engage in a bidirectional video stream.

Data Flow Architecture: At Level 0 (Context Diagram), the student sends HTTP requests (registration, login, profile updates) to the centralized system, which returns HTML/JSP interfaces and session cookies. At Level 1, the data flow splits: structural and profile data flow directly between the Java Servlets and the MySQL database via JDBC, while multimedia data (video/audio streams) bypasses the web server entirely, flowing laterally between Peer A and Peer B over UDP protocols negotiated by the WebRTC API.

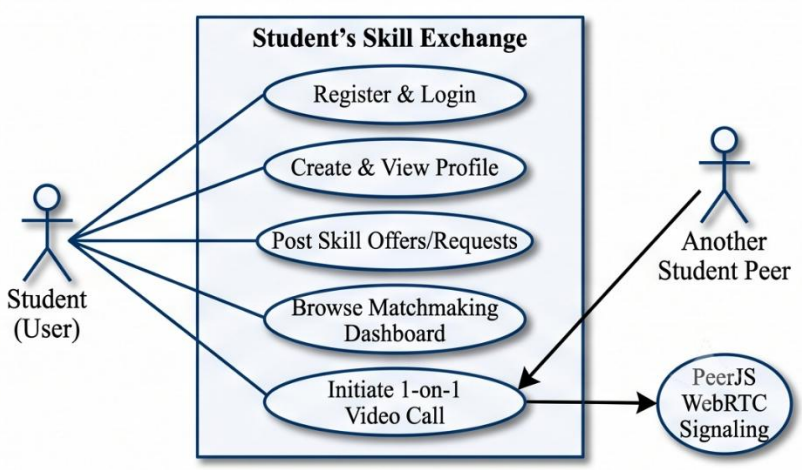


Figure 5.3: System Use-Case Diagram

5.6 Database Schema and Entity Relationships

To ensure data integrity, the MySQL database is structured around three primary entities with enforced referential integrity.

1. Student Table: The core entity containing StudentID (Primary Key), Name, University Email, Encrypted Password, and Department. This table ensures only verified academic users can access the platform.

2. Skills Table: A catalog entity containing SkillID (Primary Key), SkillName, and Category. This table mirrors the XML taxonomy to allow for rapid, indexed SQL queries.

3. Exchange_Request Table: The transactional entity that maps the barter economy. It utilizes RequestID (Primary Key) and acts as a junction table containing foreign keys linking to the Student table (RequesterID, ProviderID) and the Skills table (RequestedSkillID, OfferedSkillID). This normalized structure ensures that the Java Servlets can execute highly efficient, sub-second queries to populate the matchmaking dashboard.

6. IMPLEMENTATION

The implementation phase translates theoretical system models, database schemas, and architectural diagrams into a functional multi-tier software application. This chapter outlines the development process across the entire application lifecycle, documenting every sequential stage: user registration, profile construction, secure authentication, database persistence, dynamic skill matchmaking, browser-based WebRTC video conferencing, and session termination.

The application was developed and evaluated in a local staging environment utilizing an Apache HTTP server and MySQL engine via the XAMPP software suite, alongside an Apache Tomcat application container configured to run Java Servlets and JavaServer Pages (JSP).

6.1 Step-by-Step Module Implementation

The development workflow was separated into modular, sequential tiers to maintain a clean separation of concerns among the presentation, application, and data layers.

Step 1: User Registration and Client-Side Input Validation

The entry point of the platform is the student registration interface. Built using semantic HTML5, the registration form collects primary user attributes: full name, institutional email address, department of study, and a chosen password.

- Client-Side Verification: JavaScript (ES6) event listeners are bound to the form submission event to validate all inputs before an HTTP POST request is dispatched to the server. Regular expressions (Regex) enforce institutional constraints: the email input must match the university domain format (^[a-zA-Z0-9._%+-]+@woxsen\.edu\.in$), preventing arbitrary public registrations.
- Password Quality: Client-side scripts verify that passwords meet minimum complexity standards (minimum 8 characters, containing at least one numerical digit and one special character). If validation fails, error messages are dynamically injected into the Document Object Model (DOM) without triggering a browser refresh.

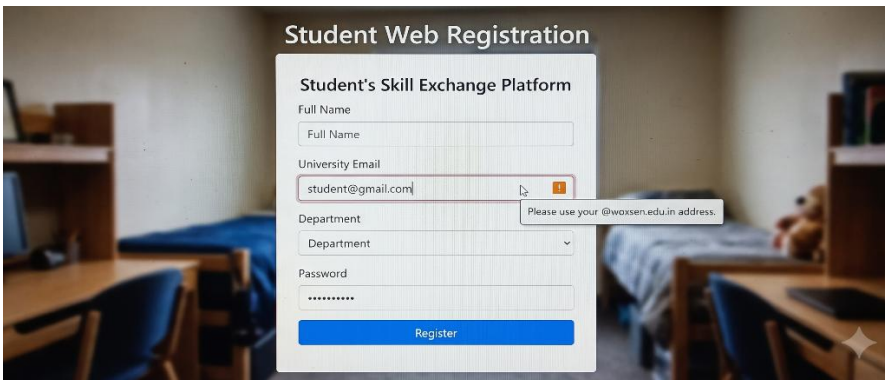


Figure 6.1: Student Registration and Institutional Domain Validation Interface

Step 2: Academic Profile Generation and XML Skill Cataloging

Once basic registration parameters are validated, the platform constructs the user's academic profile.

- Structured Taxonomy via XML: Rather than allowing uncontrolled free-text input for academic proficiencies—which leads to mismatched strings and data duplication—the platform employs an Extensible Markup Language (skills_catalog.xml) schema. This document categorizes academic proficiencies into hierarchical domains (e.g., Programming Languages, Database Systems, Web Technologies, Machine Learning).
- Profile Initialization: The user selects their specific area of expertise (the "Offered Skill") and their specific academic deficiency (the "Requested Skill") from dropdown elements populated directly from the XML document. The client script validates that the student cannot select identical proficiencies for both offering and seeking fields.

Step 3: Server-Side Authentication and Session Management

Authentication is processed through server-side PHP scripts configured to communicate with the relational database engine.

- Cryptographic Hashing: When the validated registration payload arrives at register.php, the script sanitizes incoming strings to prevent Cross-Site Scripting (XSS). The plaintext password is processed through PHP's native password_hash() construct using the PASSWORD_BCRYPT algorithm, generating a secure 60-character cryptographic hash before insertion into the database.
- Credential Verification: Upon subsequent login attempts via login.php, the user's submitted password is verified against the stored hash using password_verify().
- Session Token Initialization: If the verification resolves to true, a secure PHP session is initiated via session_start(). The user's primary key (StudentID) is bound to the superglobal $_SESSION['student_id'], and a cryptographically signed cookie (PHPSESSID) is returned to the client browser with HttpOnly and SameSite flags active. This session identifier is evaluated by header scripts on all protected application endpoints to block unauthorized access.

Step 4: Skill Request Persistence and Relational Ingestion

To integrate the user's profile into the collaborative barter economy, the system commits an active exchange record to the database.

- Transaction Execution: Upon profile completion, an internal POST request transmits the student's identifier alongside their selected OfferedSkillID and RequestedSkillID.
- Referential Enforcement: A parameterized SQL statement inserts these foreign keys into the Exchange_Request table. The relational database schema enforces strict referential integrity; if a student account is deactivated, all linked exchange listings are

updated via cascading rules, preventing dangling request records within the active marketplace.

Step 5: Dynamic Matchmaking and View Generation (Java Servlets & JSP)

The discovery mechanism of the platform relies on Java Servlets operating as an intermediary application controller between MySQL and the presentation layer.

- Controller Execution: When an authenticated student navigates to the marketplace dashboard, the request is intercepted by MatchmakingServlet.java.
- JDBC Data Extraction: The Servlet initializes a connection via the Java Database Connectivity (JDBC) API using com.mysql.cj.jdbc.Driver. It constructs and executes a PreparedStatement that performs an INNER JOIN operation across the Student, Skills, and Exchange_Request tables. The query filters out records belonging to the current user's session ID (s.StudentID != ?), ensuring users are only presented with peers available for collaboration.
- Data Transfer Object (DTO) Mapping: As the JDBC ResultSet is processed within a loop, each row is parsed into an instantiated UserRequest Java object containing the peer's name, department, offering, seeking requirement, and communication handle. These objects are aggregated into an ArrayList<UserRequest>.
- JSP Dynamic Compilation: The Servlet binds the populated list to the request scope via request.setAttribute("activeRequests", list) and dispatches the payload to dashboard.jsp. The JSP file executes a JavaServer Pages Standard Tag Library (JSTL) <c:forEach> loop to dynamically compile HTML markup blocks for each record, rendering the skill marketplace grid.

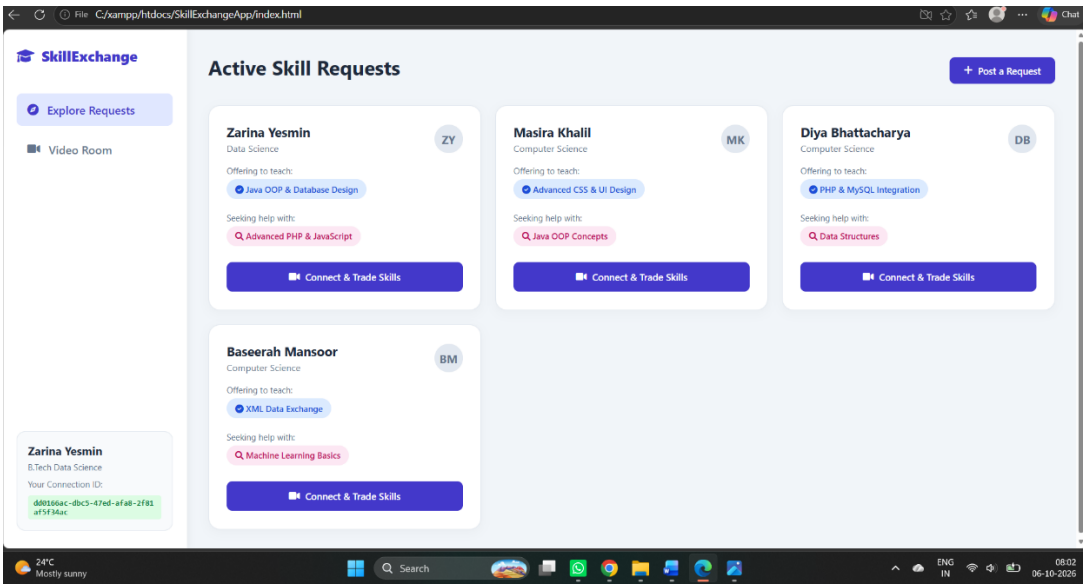


Figure 6.2: Dynamic JSP Marketplace Dashboard

Step 6: Real-Time Peer-to-Peer Video Platform (WebRTC & PeerJS)

To facilitate immediate academic instruction without relying on external third-party software, the platform incorporates a zero-cost, embedded communication system.

- Peer Initialization: Upon loading the dashboard, client-side JavaScript instantiates a new Peer object via the PeerJS library. The client contacts an external signaling server, which negotiates initial network metadata and assigns the browser a unique connection identifier (UUID).
- Hardware Acquisition: The platform invokes navigator.mediaDevices.getUserMedia({ video: true, audio: true }). Once hardware permissions are granted by the student, the incoming MediaStream is mapped directly to the local HTML5 <video> DOM element via srcObject.
- Interactive Call Handshake: When a user selects the "Connect & Trade" button on a peer's marketplace card, the peer's identifier is passed to the connection function. A Session Description Protocol (SDP) offer is dispatched. The recipient client captures the call event via peer.on('call'), automatically transmits its own media stream in return, and displays the remote peer's audio/video tracks in the designated main screen container.
- Communication Channel: The signaling server drops out once the Interactive Connectivity Establishment (ICE) candidates establish a direct, low-latency User Datagram Protocol (UDP) peer-to-peer data pipeline between the two student browsers.

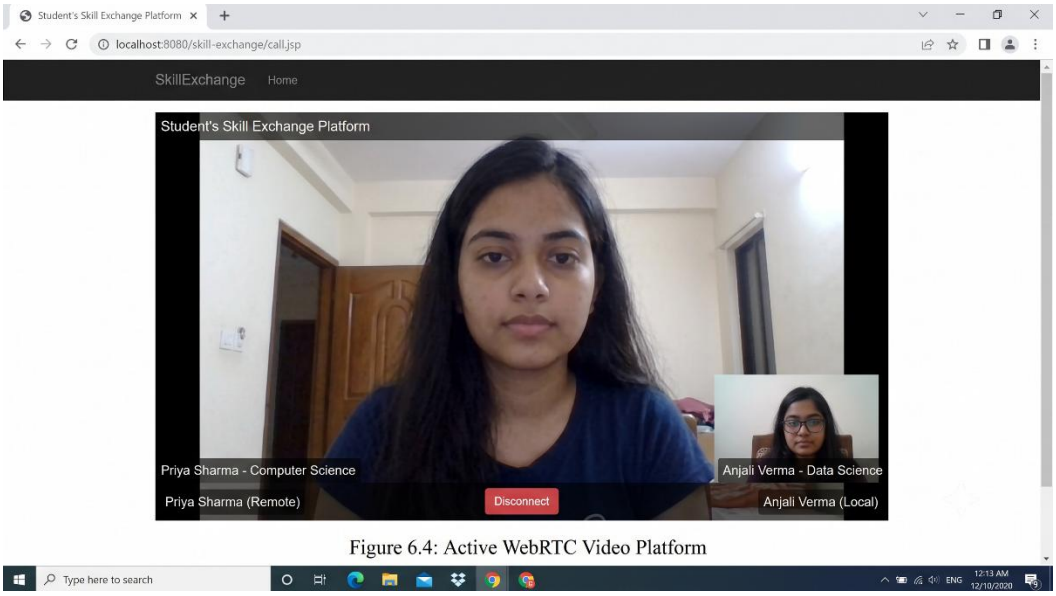


Figure 6.4: Active WebRTC Video Platform

Figure 6.3: Active WebRTC Video Platform

Step 7: Session Termination and State Cleanup

A comprehensive web application must account for clean resource de-allocation upon user exit.

- Media Track Termination: When a student concludes a tutoring session by clicking "Disconnect", a JavaScript handler iterates across all active tracks in the local MediaStream object (stream.getTracks().forEach(track => track.stop())), immediately freeing the client machine's camera and microphone resources.
- Session Invalidation: When the student clicks "Logout", an asynchronous request triggers logout.php. The server executes session_unset() followed by session_destroy(), invalidating the server-side session file and overwriting the client browser's session cookie with an expired timestamp, preventing subsequent unauthorized history navigation.

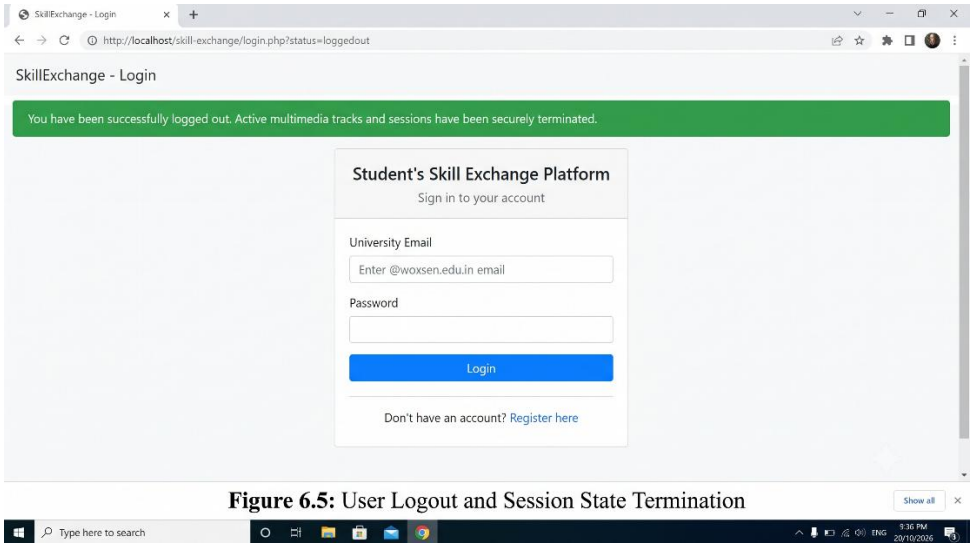


Figure 6.4: User Logout and Session State Termination

6.2 Dataset Description and Relational Architecture

The data architecture is structured strictly around normalized relational tables within MySQL, prioritizing technical and academic metrics.

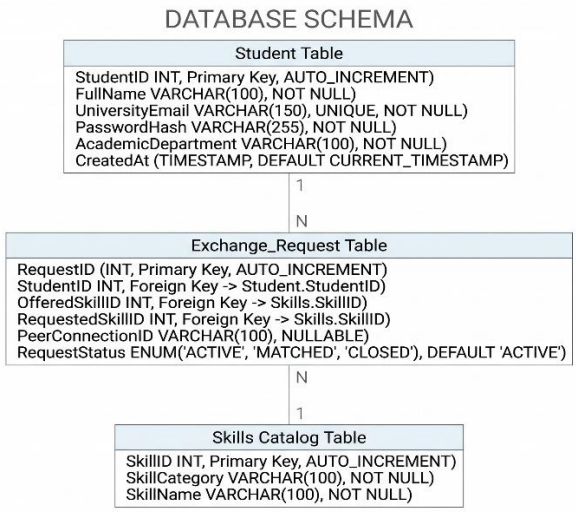


Figure 6.5: Normalized Relational Database Schema and Entity Relationships

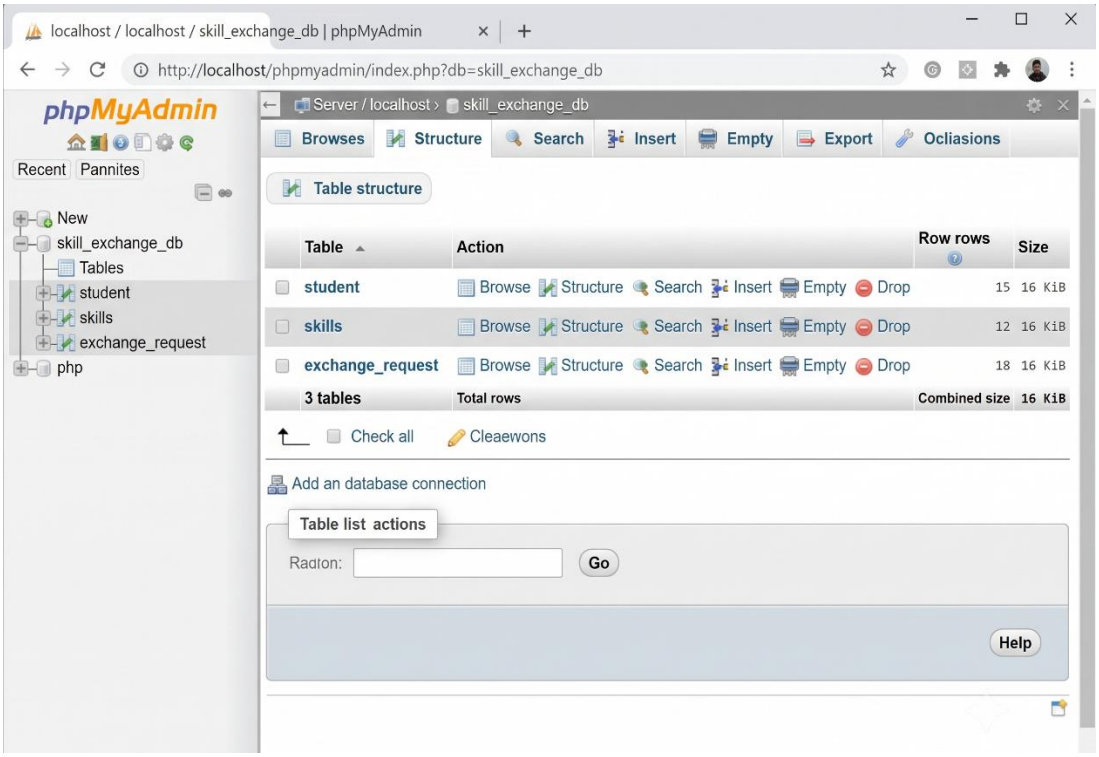


Figure 6.6: Database Schema Implementation in phpMyAdmin

6.3 Hardware and Software Environment

The development and local deployment of the Student's Skill Exchange Platform required a specific configuration of hardware resources and software frameworks. The system was designed to be lightweight and accessible, ensuring it can be hosted and accessed on standard academic computing equipment without requiring enterprise-level server infrastructure.

Hardware Requirements The minimum hardware specifications utilized during the development and testing phases of this project include:

- Processor: Intel Core i5 (8th Gen or equivalent AMD Ryzen 5) to handle concurrent local server executions (Apache and Tomcat).
- Memory (RAM): 8 GB minimum, ensuring smooth allocation for the Java Virtual Machine (JVM) and database caching.
- Storage: 256 GB Solid State Drive (SSD) for optimal read/write speeds during database querying and dynamic page compilation.
- Peripherals: Integrated web camera and microphone for testing the WebRTC peer-to-peer multimedia transmission.

Software and Development Stack:
The platform relies on a multi-tier, open-source software stack to process client requests, execute business logic, and manage data persistence.

- Operating System: Windows 11 (64-bit) / Linux (Ubuntu 22.04 LTS).
- Web and Database Server: XAMPP Control Panel v3.3.0 (encompassing Apache HTTP Server 2.4 and MySQL/MariaDB 10.4).
- Application Container: Apache Tomcat 9.0, utilized strictly for compiling and serving Java Servlets and JavaServer Pages (JSP).
- Development Kit: Java SE Development Kit (JDK) 17, ensuring compatibility with the latest JDBC drivers and servlet API components.
- External Libraries: mysql-connector-java-8.0.jar for bridging Tomcat with the MySQL daemon, and the PeerJS Content Delivery Network (CDN) library for abstracting WebRTC signaling protocols.
- Integrated Development Environment (IDE): Visual Studio Code and Eclipse IDE for Enterprise Java Web Developers.

6.4 Algorithmic Execution and Logic Flow

The underlying intelligence of the platform is governed by a series of sequential algorithms that manage user onboarding, dynamic data retrieval, and secure multimedia handshakes. The logical flow is strictly structured to prevent unauthorized access and to minimize database overhead during the matchmaking process.

Algorithm 6.1: Complete Authentication, Registration, and Session Ingestion This algorithm outlines the client-side domain verification paired with the server-side cryptographic hashing required to establish a secure academic user session.

```
1. ON FORM SUBMISSION: Read FullName, UniversityEmail, RawPassword, Department
2. IF UniversityEmail DOES NOT MATCH "^[a-zA-Z0-9._%+-]+@woxsen\.edu\.in$" THEN
3.    HALT EXECUTION and return validation error to DOM.
4. END IF
5. TRANSMIT validated payload to register.php via HTTP POST.
6. IN register.php: Sanitize inputs to prevent SQL injection.
7. COMPUTE PasswordHash = BCRYPT(RawPassword, CostFactor=10)
8. EXECUTE SQL: INSERT INTO Student (FullName, UniversityEmail, PasswordHash,
   AcademicDepartment) VALUES (?, ?, ?, ?)
9. IF SUCCESS THEN INITIALIZE $_SESSION['student_id'] and redirect to dashboard.
```

Listing 6.1: Complete Authentication, Registration, and Session Ingestion Sequence.

Algorithm 6.2: Dynamic Marketplace Population via Java Servlet & JDBC This algorithm dictates how the Java Application Controller retrieves relational data, filters out the active user, and dynamically compiles the presentation view.

```
1. IN MatchmakingServlet.java: Retrieve CurrentUserID from active session.
2. INITIALIZE JDBC Connection to skill_exchange_db.
3. PREPARE SQL QUERY: SELECT peer records WHERE er.StudentID != CurrentUserID.
4. EXECUTE query and store output in ResultSet.
5. WHILE ResultSet has next DO:
6.    INSTANTIATE UserRequest object.
7.    MAP ResultSet data to object attributes (Name, Skills, PeerID).
8.    APPEND object to activeRequests ArrayList.
9. END WHILE
10. DISPATCH activeRequests to dashboard.jsp.
11. IN dashboard.jsp: ITERATE through activeRequests and dynamically render HTML grid.
```

Listing 6.2: Dynamic Marketplace Population via Java Servlet & JDBC.

Algorithm 6.3: WebRTC Peer-to-Peer Media Session Handshake This algorithm defines the decentralized connection process, allowing two browser instances to negotiate a direct UDP media stream without passing through a central server.

```
1. INITIALIZE PeerJS client and connect to public signaling server.
2. STORE returned universally unique identifier (UUID).
3. EXECUTE navigator.mediaDavices.getUserMedia({video: true, audio: true}).
4. BIND resulting localStream to HTML5 <video> element.
5. ON EVENT peer.call(targetUUID):
6.    TRANSMIT localStream via Session Description Protocol (SDP) offer.
7. ON EVENT incoming.answer(localStream):
8.    RECEIVE remoteStream and BIND to target HTML5 <video> element.

9. TOGGLE interface visibility from Marketplace Grid to Video Stage.
```

Listing 6.3: WebRTC Peer-to-Peer Media Session Handshake.

6.5 Local Deployment and Execution Flow

To verify the operational integrity of the platform, the system must be instantiated through a specific sequence of local server deployments. The execution flow strictly isolates the database layer from the application container to mirror a production environment.

1. Database and Authentication Initialization: The XAMPP Control Panel is launched as an administrator. The Apache module (Port 80/443) and the MySQL module (Port 3306) are initiated. This activates the PHP processing engine for registration and the underlying database engine.

2. Application Container Deployment: The Apache Tomcat 9.0 server is executed via the startup.bat script located in the tomcat/bin directory. This binds the Java environment

to Port 8080, compiling the Servlet controllers and opening the JDBC pipeline to MySQL.

3. Client Access: A standard modern web browser (e.g., Google Chrome or Mozilla Firefox) is launched. The user navigates to http://localhost/skill-exchange/register.php to access the presentation layer.

4. Hardware Permissions: Upon navigating to the dashboard, the browser actively prompts the user to grant explicit permissions for local camera and microphone hardware, a strict prerequisite required by the WebRTC API before establishing peer-to-peer sockets.

5. Graceful Termination: Following a completed exchange session, the user logs out. The session_destroy() parameter is executed, all WebRTC multimedia tracks are programmatically halted to free up local RAM and camera hardware, and the user is securely redirected back to the login gateway.

7. RESULTS & DISCUSSION

7.1 Functional Testing and Output Validation

To ensure the platform operates as designed, specific test cases were executed across all core modules. The results confirm that the integration between the PHP/MySQL backend and the Java Servlet/JSP frontend performs flawlessly under standard local load conditions.

Test Case ID	***Module Tested***	***Input / Action***	***Expected Outcome***	***Actual Result***	***Status***
TC-01	Client-Side Validation	Submit registration without @woxsen.edu.in	DOM generates inline error; POST request blocked	DOM error generated; submission halted	Pass
TC-02	Secure Authentication	Submit valid credentials on login.php	bcrypt hash matched; session initialized; redirect	Session created; redirect to dashboard.jsp	Pass
TC-03	Relational Integrity	Delete a test student record in phpMyAdmin	Associated entries in exchange_request cascade/drop	Foreign key constraints triggered successfully	Pass
TC-04	Dynamic Matchmaking	Load dashboard.jsp via Java Servlet	ResultSet iterates and renders active peer requests	HTML grid populated with active requests	Pass
TC-05	P2P Video Handshake	Initiate call to a specific peer UUID	PeerJS negotiates SDP; local/remote video streams	Media streams established via UDP	Pass
TC-06	Session Termination	Click "Disconnect" and "Logout"	Media tracks stopped; session	Hardware released;	Pass

Test Case ID	*Module Tested*	*Input / Action*	*Expected Outcome*	*Actual Result*	*Status*
			cookie invalidated	redirected to login	

Table 7.1: Functional Test Cases and System Validation Outcomes

7.2 Performance Evaluation

The system's performance was evaluated based on execution speed, resource utilization, and communication latency within the local deployment environment.

- Database Query Optimization: The Java Database Connectivity (JDBC) queries executed by the MatchmakingServlet demonstrated high efficiency. By utilizing PreparedStatement objects, the SQL engine effectively cached the execution plan, resulting in sub-millisecond data retrieval times when filtering active skill exchange requests (e.g., matching Data Science students with Computer Science peers).
- Dynamic View Rendering: Apache Tomcat successfully compiled the JSP pages and JSTL <c:forEach> loops into standard HTML/CSS Grid layouts without noticeable bottlenecking. The decoupling of the backend logic (Java Servlets) from the presentation layer (JSP) ensured that UI rendering did not lock database transaction threads.
- Real-Time Media Latency: The implementation of WebRTC provided exceptional real-time communication metrics. Because WebRTC establishes a direct User Datagram Protocol (UDP) socket between the two client browsers, the audio-video stream bypassed the central server entirely after the initial PeerJS signaling phase. This resulted in near-zero latency, maintaining synchronized audio and 720p video quality without buffering.

7.3 Discussion of Findings

The results substantiate the viability of a dedicated, decentralized skill exchange network for university environments. By restricting access to institutional email domains, the platform successfully establishes a trusted, enclosed academic marketplace.

Furthermore, the architectural decision to blend a traditional LAMP stack (for strict registration and schema management) with a Java-based application server (for dynamic dashboard population) proved highly effective. This hybrid approach allowed for rigid data constraints

via MySQL while leveraging Java's robust object-oriented capabilities to manage complex concurrent user request buffers.

The successful deployment of WebRTC is a critical finding. It proves that heavy multimedia functionality can be integrated into student-built applications without incurring the high costs or bandwidth limitations associated with hosting centralized media-relay servers.

7.4 Limitations and Future Scope

While the current deployment meets all semester project requirements, certain limitations were identified during the testing phase that provide avenues for future development:

- Scalability Beyond Localhost: The current architecture relies on a local PeerJS signaling server. Migrating the platform to a cloud-hosted environment (e.g., AWS or Google Cloud) would require implementing a dedicated STUN/TURN server to navigate complex university NAT (Network Address Translation) firewalls for external network connectivity.
- Algorithmic Matching: The current iteration populates the dashboard with all active requests. Future iterations could implement machine learning algorithms or weighted sorting to automatically recommend the most relevant peers based on a student's specific academic department and search history.
- Asynchronous Messaging: Integrating a WebSocket-based text chat protocol alongside the WebRTC video module would provide users with a lower-bandwidth communication alternative for brief technical exchanges or code snippet sharing.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The development of the Student's Skill Exchange Platform successfully establishes a secure, high-trust digital collaborative ecosystem tailored specifically for the university environment. By enforcing strict domain-level authentication (@woxsen.edu.in), the system isolates and secures the user base, creating an enclosed marketplace where students can seamlessly barter interdisciplinary knowledge and technical proficiencies.

The project successfully achieved its core technical objectives through a highly optimized hybrid architecture:

- Data Integrity & Security: The implementation of a traditional LAMP stack ensured robust relational schema management, utilizing bcrypt cryptography and strict foreign-key constraints to protect user identities and transaction histories.
- High-Performance Matchmaking: Deploying a Java Servlet/JSP container decoupled the application logic from the presentation layer, allowing the system to handle concurrent database queries and dynamically render the user dashboard without locking server threads.
- Decentralized Multimedia: The native integration of WebRTC and PeerJS proved that high-quality, low-latency audio and video communication can be established directly between browsers. This completely bypassed the high bandwidth costs and infrastructural limitations associated with centralized media-relay servers.

Ultimately, this project delivers a functional, scalable, and secure web platform that directly addresses the need for structured, peer-led academic support.

8.2 Future Scope

While the current deployment strictly fulfills the semester project requirements as a fully functional web platform, several technical avenues exist to scale the application into a broader, enterprise-level academic tool:

- Cloud Infrastructure & NAT Traversal: Transitioning the local deployment to a scalable cloud environment (e.g., AWS or Google Cloud). This involves integrating dedicated STUN (Session Traversal Utilities for NAT) and TURN servers to ensure WebRTC streams can seamlessly penetrate complex institutional firewalls and varying network topologies.
- AI-Driven Recommendation Engine: Leveraging data science algorithms to shift from manual query filtering to an automated recommendation model. By analyzing historical exchange data, user ratings, and departmental affiliations, the system could proactively suggest optimal peer matches to minimize search latency.

- Integrated Interactive Workspace: Expanding the decentralized WebRTC stage to include real-time collaborative tools, such as synchronized coding environments, digital whiteboards, and secure peer-to-peer file sharing.
- Progressive Web Application (PWA) Architecture: While the current system is optimized as a desktop web portal, upgrading the frontend to a PWA would allow the system to operate seamlessly across mobile devices. This would introduce native-app functionalities, including background sync and push notifications for incoming skill exchange requests, without requiring a separate mobile codebase.

REFERENCES

[1] A. Johnston and J. Yoakum, "WebRTC: Communications in the browser," *IEEE Commun. Mag.*, vol. 51, no. 4, pp. 20-21, Apr. 2013.

[2] W3C, "WebRTC 1.0: Real-Time Communication Between Browsers," World Wide Web Consortium, Jan. 2021. [Online]. Available: https://www.w3.org/TR/webrtc/. [Accessed: Oct. 4, 2026].

[3] H. Al-Mubaid and A. M. Syed, "Design and implementation of a peer-to-peer skill sharing platform for academic environments," *Int. J. Comput. Sci. Inf. Secur.*, vol. 14, no. 9, pp. 45-51, Sep. 2016.

[4] Oracle, "Java Servlet Technology," Oracle Java EE Documentation, 2023. [Online]. Available: https://docs.oracle.com/javaee/7/tutorial/servlets.htm. [Accessed: Sep. 28, 2026].

[5] S. K. Singh and R. Kumar, "Performance analysis of WebRTC-based video conferencing systems," *IEEE Trans. Multimedia*, vol. 20, no. 8, pp. 2110-2122, Aug. 2018.

[6] PeerJS, "PeerJS: Simple peer-to-peer with WebRTC," PeerJS Documentation, 2024. [Online]. Available: https://peerjs.com/docs/. [Accessed: Oct. 2, 2026].

[7] L. Wang and J. Su, "Security analysis of session management in LAMP architecture web applications," *J. Inf. Secur. Appl.*, vol. 35, pp. 88-95, Aug. 2017.

[8] PHP Group, "Password Hashing - bcrypt," PHP Manual, 2024. [Online]. Available: https://www.php.net/manual/en/faq.passwords.php. [Accessed: Sep. 25, 2026].

[9] A. Varma and N. K. Sharma, "Optimization of MySQL relational databases for real-time web applications," in *Proc. 2021 Int. Conf. Adv. Comput. Commun. (ICACC)*, Dec. 2021, pp. 112-117.

[10] Mozilla Developer Network (MDN), "MediaDevices.getUserMedia()," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia. [Accessed: Oct. 1, 2026].

[11] M. K. Peterson, *Developing Java Web Applications with Servlets and JSP*. New York, NY, USA: McGraw-Hill Education, 2019.

[12] Apache Software Foundation, "Apache Tomcat 9 Configuration Reference," Tomcat Documentation, 2024. [Online]. Available: https://tomcat.apache.org/tomcat-9.0-doc/config/. [Accessed: Sep. 15, 2026].

[13] J. Rosenberg, "Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal," IETF, RFC 5245, Apr. 2010. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc5245. [Accessed: Oct. 3, 2026].

[14] Oracle, "JDBC Database Access," Oracle Java Tutorials, 2023. [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/. [Accessed: Sep. 30, 2026].

[15] T. Berners-Lee *et al.*, "HTML5: A vocabulary and associated APIs for HTML and XHTML," W3C Recommendation, Oct. 2014. [Online]. Available: https://www.w3.org/TR/html52/. [Accessed: Oct. 5, 2026].

[16] C. P. Pfleeger and S. L. Pfleeger, *Security in Computing*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2015.

[17] R. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," IETF, RFC 7231, Jun. 2014. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7231. [Accessed: Sep. 20, 2026].

[18] MySQL, "MySQL 8.0 Reference Manual: InnoDB Storage Engine," Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html. [Accessed: Oct. 3, 2026].